EvoCrypt: A Runtime-Upgradable Cryptographic Coprocessor for the Ethereum Virtual Machine

Anonymous Authors

Abstract. Smart contracts frequently rely on cryptographic algorithms, yet implementing complex cryptographic computation in Solidity expands it into large sequences of EVM instructions, resulting in substantial execution overhead and gas consumption. Precompiled Contracts reduce these costs by exposing client-side routines at fixed addresses, but they provide no runtime mechanism for evolving an algorithm independently of client releases and coordinated network upgrades. We present EvoCrypt, an upgradeable cryptographic coprocessor framework that exposes a stable contract-facing interface while executing replaceable WebAssembly (WASM) modules in a controlled client-side runtime. EvoCrypt distributes version metadata on-chain and deterministically activates an installed module at a designated block height, allowing participating nodes to switch algorithm versions consistently. Each invocation is charged according to a deterministic algorithm-level gas schedule associated with the activated version, thereby avoiding the cost of expanding the algorithm into EVM instructions while retaining explicit resource accounting. We implement EvoCrypt in Geth and evaluate it on a 20-node private network. The experiments show that EvoCrypt improves the efficiency of contract-facing cryptographic upgrades and invocations compared with Solidity-based implementations, while retaining near-Precompiled-Contract execution costs for most benchmark algorithms. EvoCrypt also supports the deployment of complex cryptographic algorithms that are difficult to realize directly in the EVM ecosystem.

Keywords: blockchain · smart contracts · upgrade · WebAssembly

1 Introduction

As blockchain systems become programmable platforms, smart contracts increasingly depend on cryptographic algorithms for authentication, integrity, signatures, and privacy [1–4]. The long-term security of these applications also requires the ability to adopt new algorithms, including post-quantum alternatives, without disrupting contract-facing functionality [5–7]. Blockchain clients therefore need an extensible way to provide evolving cryptographic capabilities to smart contracts.

Ethereum provides two main mechanisms for executing cryptographic operations: Solidity contract implementations and Precompiled Contracts. Solidity implementations are flexible and deployable at the contract layer, but complex cryptographic computation expands into costly EVM execution; Precompiled

Contracts execute selected routines efficiently in client-side code, but their supported algorithms are fixed to the client and normally require coordinated client updates to change [3, 4, 8].

Together, these mechanisms expose a fundamental trade-off between execution efficiency and algorithm upgradability. Solidity keeps algorithms deployable without client changes but can impose substantial execution and gas overhead, whereas Precompiled Contracts provide efficient execution only for routines embedded in the client and are therefore difficult to evolve independently. The central challenge is to replace contract-facing cryptographic implementations without sacrificing efficient execution, deterministic behavior across nodes, or compatibility with the existing EVM interface.

EvoCrypt addresses this trade-off by separating the stable contract-facing interface from replaceable WASM implementations and by separating module preparation from module activation. The interface preserves how smart contracts invoke cryptographic operations, while portable WASM modules allow implementations to be replaced without embedding each algorithm in contract bytecode or rebuilding the client for every change. Asynchronous preparation moves verification, compilation, and instantiation away from the transaction execution path. Activation-Block-based selection then ensures that nodes switch versions according to a common chain-derived rule rather than local readiness time. The design combines algorithm-level accounting with deterministic, incrementally deployable upgrades.

The main contributions of this article are summarized as follows.

- First, we design an upgradeable cryptographic coprocessor that separates the stable contract-facing interface from replaceable WASM-based algorithm implementations, allowing complex cryptographic operations to execute outside the EVM while preserving contract compatibility.
- Second, we design an activation-block-based upgrade protocol in which upgrade proposals are published on-chain, ensuring that clients derive the same proposal from a common authoritative record. By distributing and verifying a uniform WASM bytecode artifact, the protocol further ensures consistent module execution across clients when the upgrade becomes active.
- Finally, we implement EvoCrypt in Geth and evaluate it on a 20-node private blockchain network. The results show that EvoCrypt reduces the implementation burden and calling cost of the evaluated cryptographic operations while supporting dynamic evolution of contract-facing algorithms.

2 Related Work

Research on cryptographic algorithm upgrades in blockchain systems has not yet resolved the combined problem of efficient invocation and practical implementation. Related approaches can be organized by the layer at which they support upgrades: protocol and client, contract, or dynamic runtime.

Blockchain upgrades at the protocol or client layer replace execution rules through a coordinated transition. Cardano switches protocol eras with a hard-fork combinator at designated heights, and work on updatable blockchains studies how old and new chains stay synchronized [9–11]. Soft forks encode selected policy changes in rules that can be adopted without replacing the protocol [12]. Phoenix and Polkadot are closest to EvoCrypt in supporting live replacement: Phoenix compiles upgrade code carried in transactions at runtime, while Polkadot replaces governance-approved on-chain WASM runtime logic during block synchronization [13, 14]. Both nevertheless replace client or chain-runtime components rather than a contract-facing cryptographic algorithm. They do not specify how a contract names an algorithm, binds its interface, or switches versions without redeploying the caller. EvoCrypt narrows the replacement boundary to individual WASM modules behind a stable Solidity-facing interface, with consistency derived from hash-identified metadata, activation height, and per-algorithm resource parameters.

Contract-level schemes update application logic without replacing the entire blockchain client, using deployable contracts, proxy indirection, or bytecode rewriting. EVMPatch patches deployed Ethereum smart contracts through bytecode rewriting and proxy-based deployment, while Solidity implementations offer application-level flexibility within the EVM [15, 4]. LEAGAN extends contract-level upgradeability through data separation, incremental hashing, revision control, and status-contract-based version management [16]. Precompiled Contracts provide a complementary client-side path for selected cryptographic routines through reserved EVM-facing addresses [8, 17, 18]. Contract-level implementations remain constrained by EVM execution cost and state or interface compatibility, whereas adding or replacing a Precompiled Contract normally requires changes to client and protocol rules.

Dynamic software-update schemes move replacement below the application layer by loading new components into a running host. KylinX supports runtime library replacement through dynamic mapping, Mvedsua combines dynamic updates with multi-version execution, PYLIVE changes Python code online, and distributed-system research addresses modular updates and version consistency [19–23]. WASM provides a portable artifact format and explicit host boundary for this style of replacement. In blockchain systems, existing WASM-based upgrades generally target broad runtime logic rather than a narrow EVM-facing cryptographic interface [24, 14].

3 EvoCrypt Design

EvoCrypt is a cryptographic coprocessor framework for dynamically upgrading contract-facing cryptographic algorithms. This section presents the system model, the upgrade protocol, and the invocation protocol that together define the design. We focus on the coordination, version-selection, and contract-facing execution rules of EvoCrypt, while the prototype implementation and experimental configuration are described in Section 5.

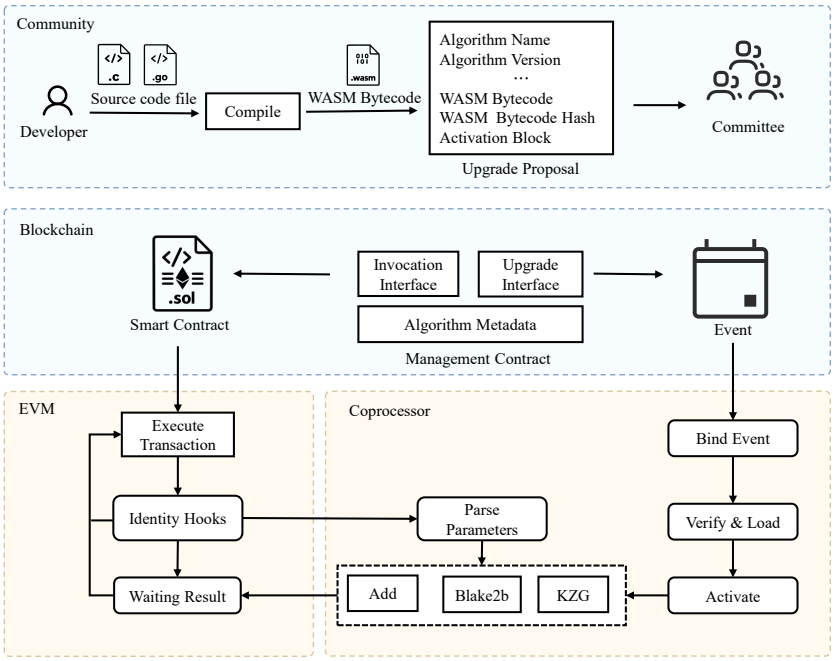


Fig. 1. System model for proposed EvoCrypt framework.

3.1 System Model

EvoCrypt separates governance, chain state, contract execution, and module execution into four cooperating components: the Community, the Blockchain, the EVM, and the Cryptographic Coprocessor. The Community includes developers and an authorizing committee that proposes and approves algorithm upgrades. The Blockchain maintains the canonical chain and shared world state, including smart contracts and the Management Contract. The EVM processes smart-contract transactions, while the node-local Cryptographic Coprocessor prepares, activates, and executes replaceable cryptographic modules.

Two chain-derived paths connect these components. An approved proposer records an upgrade through the Management Contract, whereas a contract initiates an invocation through the EVM of a Blockchain node. The Management Contract stores algorithm metadata and exposes the contract-facing upgrade and invocation interfaces. The coprocessor uses the resulting chain state to prepare and execute the corresponding module. Figure 1 summarizes these objects and their relationships.

3.2 Upgrade Protocol

On-chain publication. On-chain publication provides the authoritative record that coordinates node-local preparation and height-based activation. After the

committee approves a proposal, a committee-authorized proposer submits it through the designated upgrade interface. The Management Contract admits the proposal only after the committee has reached consensus on its contents, stores the version metadata, and emits an upgrade event.

The EVM records metadata but does not compile or instantiate the WASM module. Node-dependent preparation therefore remains outside the transaction that changes the chain state. Every node reconstructs the same version from the admitted record, which identifies the algorithm, version, WASM artifact, module hash, interface descriptors, resource parameters, and Activation Block. Only $\langle a, v, H_{a,v} \rangle$ is emitted as a notification index. Let $\mathcal{V}_a$ denote the versions already registered for algorithm a; the authoritative record prevents nodes from preparing different artifacts or execution rules for the same upgrade.

Definition 1 (Version metadata). *Let a be an algorithm name and v a version number. An admitted upgrade is the metadata record*

$$\mu_{a,v} = \big(a,\, v,\, C_{a,v},\, h_{a,v},\, g^{(0)}_{a,v},\, \lambda_{a,v},\, \tau^{\text{in}}_{a,v},\, \tau^{\text{out}}_{a,v},\, H_{a,v}\big), \tag{1}$$

where $C_{a,v}$ is the compiled WASM bytecode, $h_{a,v} = \text{Keccak256}(C_{a,v})$ is the module hash, $g^{(0)}_{a,v}$ is the initial gas fee, $\lambda_{a,v}$ is the gas slope parameter, $\tau^{\text{in}}_{a,v}$ and $\tau^{\text{out}}_{a,v}$ are the ABI input and output type descriptors, and $H_{a,v}$ is the Activation Block height. The Management Contract stores an encoding of these fields. Each node recomputes $h_{a,v}$ from the recovered bytecode during preparation and binds that hash to the prepared version.

Node-local preparation. Node-local preparation converts the admitted metadata record into executable state without changing the active version. After observing the upgrade event, each client recovers $\mu_{a,v}$, checks that the event index agrees with the stored record, decodes the artifact, and recomputes its hash. The Upgrade Handler then compiles and instantiates the module and validates its required entry point. Once these checks succeed, the node records (a, v) as prepared. Algorithm 1 gives the complete publication and preparation procedure.

Preparation is deliberately separated from the chain-level upgrade record. Node-local work can proceed asynchronously, failed attempts can retry from the authoritative chain record, and an executed upgrade transaction is not rolled back. The separation allows preparation to complete without changing the version selected by the chain.

Height-based activation. Height-based activation makes the selected version a function of shared chain height rather than node-local preparation time. The Upgrade Block includes the admitted upgrade transaction and emits $\langle a, v, H_{a,v} \rangle$. After that block, each node independently completes the off-chain Load–Verify–Ready sequence, while invocations before $H_{a,v}$ continue to use the previously selected version. At the Activation Block, nodes with the same chain view select version v according to the rule below. Compilation-time differences therefore cannot change the version selected at a given execution height.

When a selected version is not yet prepared, the node reloads its authoritative metadata and continues preparation rather than selecting an obsolete version.

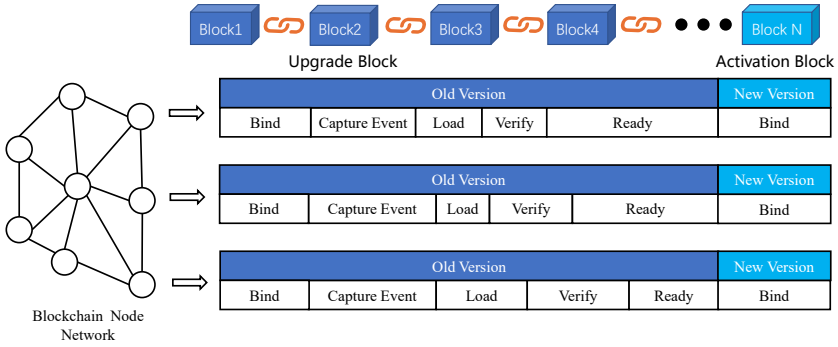


Fig. 2. Node-local preparation and deterministic activation of an EvoCrypt upgrade.

Algorithm 1 Activation-Block-based WASM upgrade protocol.

Require: Metadata $\mu_{a,v}$ as in (1); node set $\mathcal{N}$
Ensure: Prepared set updated on every $n_i \in \mathcal{N}$

committee-authorized proposer submits $\mu_{a,v}$ through the designated upgrade interface
Management Contract stores $\mu_{a,v}$; emits event $\langle a, v, H_{a,v} \rangle$
for all $n_i \in \mathcal{N}$ **do**
    observe $\langle a, v, H_{a,v} \rangle$; fetch $\mu'_{a,v}$ from Management Contract
    if event fields disagree with $\mu'_{a,v}$ **then**
        reject (a, v)
    else
        decode $\widehat{C}$ from $\mu'_{a,v}$; reject (a, v) unless Keccak256$(\widehat{C}) = h_{a,v}$
        repeat
            compile and instantiate $\widehat{C}$; validate the required module entry point
        until preparation succeeds
        persist $\widehat{C}$ under $h_{a,v}$ with $\mu_{a,v}$; mark (a, v) prepared
    end if
end for

The policy preserves height-based selection while allowing node-local readiness to complete asynchronously. Figure 2 shows the separation between publication, preparation, and activation.

Definition 2 (Active version). *Let $\mathcal{V}_a$ be the registered version set of algorithm a, and let b be the execution block height. The version used at height b is*

$$v^*(a, b) = \arg\max_{v \in \mathcal{V}_a} \{(H_{a,v},\, v) \mid H_{a,v} \le b\}\,. \tag{2}$$

The arg max *is lexicographic on the pair $(H_{a,v}, v)$: among versions whose activation height does not exceed b, the rule prefers the latest activation height and, for a tied height, the larger version number. If the feasible set is empty, no version is active and the invocation fails with a missing-version error.*

3.3 Invocation Protocol

A unified Management Contract entry point keeps the contract-facing call boundary stable. The caller serializes the algorithm name a and its parameters into an encoded request x and submits it through the designated interface, so different algorithms share one destination address and the contract binds to the ABI descriptors of the selected version rather than to module location, implementation language, or node-local loading state. EVM-side Hooks admit only requests to that interface; all others remain on the ordinary EVM path. For a routed request the coprocessor parses (a, x), resolves $v^*(a, b)$ by (2), and, under the mandatory-preparation policy, prepares the selected version if necessary rather than falling back to an obsolete implementation. The selected module receives the encoded parameters and returns an encoded result; the call shape remains compatible when $\tau^{\text{in}}_{a,v}$ and $\tau^{\text{out}}_{a,v}$ are compatible. Algorithm 2 specifies this procedure and its protocol-error conditions.

Definition 3 (Algorithm-level gas schedule). *Each algorithm version uses an independent deterministic pricing rule. A call with inner input x against version v of algorithm a is charged*

$$G(a, v, x) = g^{(0)}_{a,v} + \lambda_{a,v} \cdot |x|, \tag{3}$$

where $|x|$ is the byte length of the call-stage inner input, $g^{(0)}_{a,v}$ is the initial gas fee, and $\lambda_{a,v}$ is the gas slope parameter registered in $\mu_{a,v}$.

Algorithm 2 Contract-to-coprocessor invocation protocol.

Require: Recognized invocation request d, block height b, gas budget g
Ensure: ABI-encoded result y or explicit error
1: $(a, x) \leftarrow \text{Parse}(d)$
2: **if** the request is invalid **then**
3: **return** ABI error
4: **end if**
5: $v \leftarrow v^*(a, b)$ by (2)
6: **if** no applicable version exists **then**
7: **return** missing-version error
8: **end if**
9: ensure (a, v) is prepared under the mandatory-preparation policy
10: $G_{\text{req}} \leftarrow G(a, v, x)$ by (3)
11: **if** $g < G_{\text{req}}$ **then**
12: **return** out-of-gas error
13: **end if**
14: charge G_{req}; $y \leftarrow \text{Execute}(a, v, x)$
15: **return** y, or runtime error on failure

Because G depends only on registry coefficients and $|x|$, every node that resolves the same $v^*(a, b)$ and receives the same input charges an identical fee,

independently of local CPU time or WASM instruction counts. Coefficients used in Section 5 demonstrate the pricing mechanism rather than a calibrated economic optimum.

4 Security Analysis

This section formalizes the upgrade protocol and derives the invocation guarantees from it. Section 4.1 fixes the ledger and registry abstractions, the trusted assumptions, the adversary, and game-based definitions for the two failure modes an upgrade mechanism must exclude: honest nodes selecting different versions for the same height, and an honest node executing bytecode other than the admitted artifact. Section 4.2 shows that EvoCrypt excludes the first unconditionally relative to the ledger abstraction and the second by a tight reduction to the collision resistance of Keccak256, and combines both into an upgrade-soundness theorem. Section 4.3 extends consistency to invocation outcomes, and Section 4.4 treats the residual risks the model leaves open and the governed recovery path that addresses them.

4.1 Security Model

Notation and ledger. κ is the security parameter, $\mathsf{negl}(\kappa)$ a negligible function, and algorithms are probabilistic polynomial time (PPT) unless stated otherwise. $\mathcal{N}$ is the set of honest nodes, each running Algorithms 1 and 2. The symbols $\mu_{a,v}$, $C_{a,v}$, $h_{a,v}$ and $H_{a,v}$ are as in Definition 1, and $u_{a,v}$ denotes the height of the Upgrade Block that admitted $\mu_{a,v}$. We abstract the blockchain as a transaction ledger: honest node n holds a local view $\mathcal{L}_n$, a finite block sequence with prefix $\mathcal{L}_n[1..b]$ and height $|\mathcal{L}_n|$. Transaction order inside a block is part of the ledger, so an execution point is a height together with a position in the block; we write b for both.

Definition 4 (Registry state). *The registry* $\mathsf{Reg}(P)$ *of a ledger prefix* P *is the Management Contract storage obtained by executing* P *from the genesis state; by EVM determinism it is a function of* P*. For a node* n *and height* b*, let* $\mathcal{V}_a^n(b) = \{v \mid \mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n[1..b])\}$ *and let* $v_n^*(a,b)$ *be the value of* (2) *over* $\mathcal{V}_a^n(b)$*, with* $v_n^*(a,b) = \perp$ *when the feasible set is empty.*

Definition 4 fixes the version each honest node selects from its ledger prefix. Agreement on that version, and execution of the admitted artifact, both depend on assumptions that we now make explicit. Assumptions 1–3 are used for upgrade soundness in Section 4.2, and Assumption 4 only when Section 4.3 extends consistency to invocation outcomes.

Assumption 1 (Canonical ledger). *For all honest* $n, n' \in \mathcal{N}$ *and every* $b \leq \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$*,* $\mathcal{L}_n[1..b] = \mathcal{L}_{n'}[1..b]$.

This is the persistence property of a robust transaction ledger [25], applied to the prefixes at which invocations are evaluated; blocks still subject to reorganization are re-executed by the ordinary EVM rules and lie outside the model.

Assumption 2 (Committee-gated admission). *The Management Contract admits $\mu_{a,v}$ only on a committee-authorized submission, and every admitted record is well formed: (i) $h_{a,v} = \mathrm{Keccak256}(C_{a,v})$; (ii) $u_{a,v} < H_{a,v}$, i.e., the record is committed strictly before its Activation Block; (iii) (a, v) is unique, and an admitted record is never modified or removed; (iv) $C_{a,v}$ passes validation of the configured WASM Runtime. The committee is further trusted to review the implementation and its gas schedule before admission. Conditions (i)–(iii) can be enforced by the contract itself; listing them as assumptions keeps the analysis independent of the contract implementation.*

Assumption 3 (Collision resistance). *For every PPT algorithm $\mathcal{B}$,*

$$\mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}) = \Pr\big[(C, C') \leftarrow \mathcal{B}(1^\kappa) : C \neq C' \wedge \mathrm{Keccak256}(C) = \mathrm{Keccak256}(C')\big] \leq \mathsf{negl}(\kappa). \tag{4}$$

Keccak256 is unkeyed, so (4) is read in the concrete sense of Rogaway [26]: each reduction below exhibits an explicit collision finder $\mathcal{B}$ and bounds the protocol adversary's advantage by that of $\mathcal{B}$.

Assumption 4 (Deterministic runtime). *The WASM Runtime exposes a validation predicate $\mathsf{Validate}(C) \in \{0, 1\}$ and an execution function $\mathsf{Exec}(C, x)$ whose outputs, including error outcomes, depend only on (C, x) and the fixed, allowlisted host interface. It enforces bounded linear memory, import validation and the configured resource limits, and exposes no host state or capability beyond that interface.*

Adversary. The PPT adversary $\mathcal{A}$ may (a) submit arbitrary transactions, including invocation inputs and candidate upgrade proposals, the latter admitted only subject to Assumption 2; (b) schedule the delivery of upgrade events, delay or interrupt node-local preparation, and crash and restart nodes so that preparation retries; and (c) tamper with the node-local artifact path, i.e., replace the byte sequence $\widehat{C}$ that a node obtains when it decodes, persists or reloads an artifact, modelling corrupted caches, persisted copies and untrusted retrieval channels. It cannot violate Assumptions 1–3, forge committee authorization, or alter the client binary. The model thus excludes a compromised Geth binary, a broken consensus protocol, a captured committee, a failed implementation review, and a primitive whose underlying assumption is false.

Security definitions. Each experiment below runs $\mathcal{A}$ against the honest nodes in this model and outputs 1 when $\mathcal{A}$ causes the undesirable event; the advantage of $\mathcal{A}$ is the probability of output 1 over all coins.

Definition 5 (Version divergence). *In $\mathsf{Exp}^{\mathrm{div}}_{\mathcal{A}}(\kappa)$, the adversary finally outputs a tuple (a, b, n, n') with $n, n' \in \mathcal{N}$ and $b \leq \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$. The experiment outputs 1 iff $v^*_n(a, b) \neq v^*_{n'}(a, b)$, where $\perp$ differs from every version.* $\mathsf{Adv}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = 1]$.

Definition 6 (Module substitution). $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ *outputs* 1 *iff some honest node* n *marks* (a, v) *as prepared with, or executes for* (a, v)*, a byte sequence* $\widehat{C}$ *such that* $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$ *and* $\widehat{C} \neq C_{a,v}$*.* $\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) = 1]$.

Definition 7 (Upgrade soundness). $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$ *outputs* 1 *iff some honest node executes an invocation of* a *at height* b *with a byte sequence* $\widehat{C} \neq C_{a,v^*(a,b)}$*, where* $v^*(a, b)$ *is evaluated on the canonical prefix of height* b*; executing any module when* $v^*(a, b) = \perp$ *also counts as output* 1*. EvoCrypt is* upgrade sound *if* $\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) = 1]$ *is negligible for every PPT* $\mathcal{A}$*.*

Definition 7 is the target property: whatever $\mathcal{A}$ does to scheduling and node-local storage, every honest node that answers an invocation runs exactly the artifact the canonical chain designates for that height. Definitions 5 and 6 isolate the two ways this can fail.

4.2 Upgrade Security

Lemma 1 (Activation timeliness). *Under Assumption 2, for every honest node* n *and every* $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$*: (i)* $v^*_n(a, b) \neq v$ *for every* $b < H_{a,v}$*, and (ii)* v *lies in the feasible set of* (2) *at every* b *with* $H_{a,v} \leq b \leq |\mathcal{L}_n|$*.*

Proof. (i) The feasible set contains only versions with $H_{a,v} \leq b$. (ii) By Assumption 2(ii), $u_{a,v} < H_{a,v} \leq b$, so the admitting transaction lies in $\mathcal{L}_n[1..b]$, and by (iii) the record is never removed; hence $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n[1..b])$ and $H_{a,v} \leq b$ places v in the feasible set. □

The lemma confines the preparation window of (a, v) to heights $u_{a,v} < b < H_{a,v}$ and excludes retroactive activation: appending a block never changes the version designated for an already-executed height, which is why a node replaying history, such as the late-joining node in Section 5.3, agrees with nodes that executed it live.

Theorem 1 (Activation consistency). *Under Assumptions 1 and 2, every adversary* $\mathcal{A}$*, including a computationally unbounded one, has* $\mathsf{Adv}^{\mathrm{div}}_{\mathcal{A}}(\kappa) = 0$*.*

Proof. Fix any output (a, b, n, n') of $\mathcal{A}$. By Assumption 1, $P = \mathcal{L}_n[1..b] = \mathcal{L}_{n'}[1..b]$, and by Definition 4 $\mathsf{Reg}(P)$ is a function of P, so both nodes hold the same records and the same feasible set $\{(H_{a,v}, v) : H_{a,v} \leq b\}$. By Assumption 2(iii) its elements are pairwise distinct, so the lexicographic maximum in (2) is unique when the set is non-empty, and both nodes return $\perp$ when it is empty; hence $v^*_n(a, b) = v^*_{n'}(a, b)$. No adversarial capability enters this computation: scheduling, preparation timing, crashes and artifact tampering act on node-local state, whereas line 5 of Algorithm 2 evaluates (2) on the registry before any node-local state is consulted. The experiment therefore never outputs 1, whatever the running time of $\mathcal{A}$. □

Theorem 1 is information-theoretic. The next theorem shows that binding the executed bytecode to the registry costs exactly one collision-resistance assumption.

Theorem 2 (Module binding). *Under Assumption 2, for every PPT adversary $\mathcal{A}$ there exists a PPT algorithm $\mathcal{B}$, running in the time of $\mathcal{A}$ plus the cost of simulating the honest nodes, such that*

$$\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{5}$$

In particular, $\mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ is negligible under Assumption 3.

Proof. On input 1^κ, $\mathcal{B}$ runs $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ internally: it simulates the ledger and every honest node, executing Algorithms 1 and 2 on their behalf, and answers the scheduling and artifact-tampering requests of $\mathcal{A}$ exactly as the real experiment would. The simulation is perfect because the model contains no secret state. Whenever a simulated honest node is about to mark (a, v) prepared with $\widehat{C}$ (Algorithm 1, line 12) or to execute $\widehat{C}$ for (a, v) (Algorithm 2, line 14), $\mathcal{B}$ reads $C_{a,v}$ from the simulated registry; if $\widehat{C} \neq C_{a,v}$ it outputs $(\widehat{C}, C_{a,v})$ and halts, and otherwise it eventually outputs $\perp$.

Let E be the event that $\mathsf{Exp}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ outputs 1; by perfect simulation, $\Pr[E] = \mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa)$ in the run of $\mathcal{B}$. On E, an honest node accepted $\widehat{C}$ for (a, v), which happens only after verifying $\mathrm{Keccak256}(\widehat{C}) = h_{a,v}$ against the canonical record (Algorithm 1, line 8; the prepared set is keyed by $h_{a,v}$, so execution reloads the artifact under the same check). By Assumption 2(i), $h_{a,v} = \mathrm{Keccak256}(C_{a,v})$, so $(\widehat{C}, C_{a,v})$ is a collision and $\Pr[E] \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$, which is (5). The reduction is tight: it loses no advantage and does not guess the target record. □

The hash binds only the bytecode; ABI descriptors, gas parameters and Activation Block are read directly from the canonical registry, which Theorem 1 already fixes across honest nodes.

Theorem 3 (Upgrade soundness). *Under Assumptions 1–3, EvoCrypt is upgrade sound: for every PPT adversary $\mathcal{A}$ there exists a PPT $\mathcal{B}$ as in Theorem 2 such that*

$$\mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{6}$$

Proof. Let W be the event that $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$ outputs 1 at honest node n for an invocation of a at height b executed with $\widehat{C}$; let $v = v^*(a, b)$ be the canonical selection, v_n the version resolved by n in line 5 of Algorithm 2, and $\hat{v}$ the version under which $\widehat{C}$ was prepared. Then $W \subseteq E_1 \cup E_2 \cup E_3$ with

$$E_1 : v_n \neq v, \qquad E_2 : v_n = v \wedge \hat{v} \neq v_n, \qquad E_3 : \hat{v} = v_n = v \wedge \widehat{C} \neq C_{a,v},$$

since if none occurs then $\widehat{C} = C_{a,v^*(a,b)}$. $\Pr[E_1] = 0$: node n evaluates (2) on $\mathsf{Reg}(\mathcal{L}_n[1..b])$, which by Assumption 1 is the canonical prefix, so the argument of Theorem 1 gives $v_n = v$; if $v = \perp$ the node returns a missing-version error (line 7) and executes nothing. $\Pr[E_2] = 0$: line 9 admits execution only for the resolved pair (a, v_n) and, under the mandatory-preparation policy, continues preparing (a, v_n) rather than falling back to another prepared version, so the only artifact reaching line 14 was prepared for (a, v_n). E_3 is a module substitution

for (a, v) with $\mu_{a,v} \in \mathsf{Reg}(\mathcal{L}_n)$, the latter by Lemma 1(ii) since $H_{a,v} \le b$; hence $\Pr[E_3] \le \mathsf{Adv}^{\mathrm{sub}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$ by Theorem 2. A union bound yields (6). □

Remark 1 (Safety versus liveness). Theorems 1–3 are safety statements: an honest node never selects or runs a version other than the canonical one. They do not assert that every node can answer at height $H_{a,v}$; a node still preparing, or in the retry loop of Algorithm 1, defers rather than degrades. Under Assumption 2(iv) and the deterministic validation of Assumption 4, an admitted artifact validates identically at every honest node, so preparation cannot succeed at one node and fail permanently at another; how quickly it completes is measured in Sections 5.2 and 5.3 rather than proven here.

4.3 Invocation Security

Section 4.2 fixes *which* artifact an honest node executes. This subsection shows that, once Assumption 4 is added, agreement on the artifact lifts to agreement on the invocation outcome and on the resulting EVM state transition. Assumption 4 is the only new ingredient; it is an engineering assumption on the configured WASM Runtime rather than a cryptographic one, and the statements below make its role explicit. An *outcome* of Algorithm 2 is the pair (o, γ), where o is either the ABI-encoded result y or one of the protocol errors returned by the algorithm, and γ is the gas charged, with $\gamma = 0$ when the algorithm returns before line 14.

Definition 8 (Invocation divergence). *In* $\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa)$*, the adversary finally outputs* (d, g, b, n, n') *with* $n, n' \in \mathcal{N}$ *and* $b \le \min(|\mathcal{L}_n|, |\mathcal{L}_{n'}|)$*, and both nodes run Algorithm 2 on request* d *with gas budget* g *at height* b*. If either node has not produced an outcome, the experiment outputs* 0*; otherwise it outputs* 1 *iff* $(o_n, \gamma_n) \neq (o_{n'}, \gamma_{n'})$*.* $\mathsf{Adv}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) = \Pr[\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) = 1]$.

Returning 0 when a node has not answered keeps the definition a safety property, in line with Remark 1: a node still preparing at line 9 defers and is not counted as disagreeing.

Lemma 2 (Pre-execution agreement). *Under Assumptions 1 and 2, two honest nodes running Algorithm 2 on the same* (d, g, b) *take the same branch at every line before line 14, resolve the same version* $v = v^*(a, b)$*, and compute the same* $G_{\mathrm{req}} = G(a, v, x)$*. In particular, if either node returns an ABI, missing-version or out-of-gas error, both return the same error with* $\gamma = 0$*.*

Proof. Parse is a fixed function of d, so both nodes obtain the same (a, x) or the same ABI error. Theorem 1 gives $v^*_n(a, b) = v^*_{n'}(a, b)$, so the missing-version test agrees. If a version is resolved, both nodes read $\mu_{a,v}$ from $\mathsf{Reg}(\mathcal{L}_n[1..b]) = \mathsf{Reg}(\mathcal{L}_{n'}[1..b])$ (Assumption 1 with Definition 4), hence the same $g^{(0)}_{a,v}$, $\lambda_{a,v}$, $\tau^{\mathrm{in}}_{a,v}$ and $\tau^{\mathrm{out}}_{a,v}$. Since $|x|$ is determined by d, (3) yields the same G_{req}, and comparison with the common budget g yields the same out-of-gas decision. None of these

steps reads node-local state other than the prepared set, which only decides whether line 9 must prepare first, not which branch is taken. □

Theorem 4 (Invocation consistency). *Under Assumptions 1–4, for every PPT adversary $\mathcal{A}$ there exists a PPT $\mathcal{B}$ as in Theorem 2 such that*

$$\mathsf{Adv}^{\mathrm{inv}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B}). \tag{7}$$

Moreover, whenever both nodes answer, they charge the same gas unconditionally: $\gamma_n = \gamma_{n'} = G(a, v^*(a,b), x)$ if line 14 is reached and $\gamma_n = \gamma_{n'} = 0$ otherwise.

Proof. Let W be the event that $\mathsf{Exp}^{\mathrm{inv}}_{\mathcal{A}}(\kappa)$ outputs 1, so both nodes answered and $(o_n, \gamma_n) \neq (o_{n'}, \gamma_{n'})$. Let $v = v^*(a,b)$, and let $\widehat{C}_n, \widehat{C}_{n'}$ be the byte sequences the nodes execute at line 14 when they reach it. Then $W \subseteq F_1 \cup F_2 \cup F_3$ with

$$\begin{aligned} F_1 &: \text{the nodes take different branches before line 14,} \\ F_2 &: \text{both reach line 14 and } \widehat{C}_n \neq C_{a,v} \ \vee \ \widehat{C}_{n'} \neq C_{a,v}, \\ F_3 &: \widehat{C}_n = \widehat{C}_{n'} = C_{a,v} \ \wedge \ o_n \neq o_{n'}, \end{aligned}$$

since if none occurs, both nodes either return the same pre-execution error with $\gamma = 0$, or reach line 14 with the same G_{req}, execute the same bytecode and return the same o. $\Pr[F_1] = 0$ by Lemma 2. F_2 states that some honest node executes an invocation of a at height b with bytecode other than $C_{a,v^*(a,b)}$, which is exactly the winning event of $\mathsf{Exp}^{\mathrm{bind}}_{\mathcal{A}}(\kappa)$; hence $\Pr[F_2] \le \mathsf{Adv}^{\mathrm{bind}}_{\mathcal{A}}(\kappa) \le \mathsf{Adv}^{\mathrm{cr}}_{\mathrm{Keccak256}}(\mathcal{B})$ by Theorem 3. The bound covers both nodes at once because $\mathsf{Exp}^{\mathrm{bind}}$ already quantifies over all honest nodes. $\Pr[F_3] = 0$: on F_3 both nodes evaluate $\mathsf{Exec}(C_{a,v}, x)$ on the same arguments, and by Assumption 4 its output, including any runtime error, is a function of $(C_{a,v}, x)$ and the fixed host interface; ABI encoding of that output under the common $\tau^{\mathrm{out}}_{a,v}$ is deterministic, so $o_n = o_{n'}$. A union bound gives (7). The gas statement is Lemma 2: γ equals G_{req} when line 14 is reached and 0 otherwise, and both quantities agree without any computational assumption. □

The bound in (7) coincides with that of Theorem 3: adding Assumption 4 introduces no further probabilistic loss, and the charged gas is agreed unconditionally. The latter is the formal counterpart of Definition 3: because G depends only on registry fields and $|x|$, no node-local measurement enters the fee, and differences in CPU time or in the number of WASM instructions executed cannot make honest nodes charge differently.

4.4 Residual Risks and Governed Recovery

Two risks remain inside the model of Section 4.1 after Theorems 1–4: an admitted implementation may later prove unsuitable or vulnerable, and an admitted gas schedule may underprice or overprice the resources a call consumes. Neither is excluded by the committee review of Assumption 2, which is trusted but

not infallible. EvoCrypt does not prevent such admissions; it guarantees that a governed correction, once admitted, takes effect at every honest node and renders the faulty version unreachable.

Corollary 1 (Governed supersession). *Under Assumptions 1 and 2, let $\mu_{a,v}$ and $\mu_{a,v'}$ be admitted with $H_{a,v'} > H_{a,v}$. Then for every honest node n and every height $H_{a,v'} \le b \le |\mathcal{L}_n|$, $v_n^*(a,b) \neq v$. If, moreover, no admitted version of a other than v' has an activation height in $[H_{a,v'}, b]$, then $v_n^*(a,b) = v'$.*

Corollary 1 covers the three corrective actions. *Replacement* admits $\mu_{a,v'}$ with new bytecode; from $H_{a,v'}$ on, Theorem 3 guarantees that every honest node runs $C_{a,v'}$ and never $C_{a,v}$. *Repricing* admits $\mu_{a,v'}$ with revised $g^{(0)}_{a,v'}, \lambda_{a,v'}$ and possibly unchanged bytecode, which Assumption 2(iii) permits because uniqueness is over (a,v) rather than over $h_{a,v}$; Lemma 2 then charges the new fee unconditionally. *Deprecation* admits a version whose module returns a protocol error on every input, so that by Theorem 4 all invocations of a fail deterministically after $H_{a,v'}$ without any contract redeployment. In each case, the correction becomes effective exactly $H_{a,v'} - u_{a,v'}$ blocks after admission, and a node is ready at that height whenever its preparation completes within this window (Remark 1); Section 5.2 measures the preparation time that this window must cover.

5 Evaluation

This evaluation asks whether EvoCrypt makes upgradeable cryptographic algorithms practical for contract-facing use. We measure the cost of making an algorithm callable and the steady-state invocation cost, comparing the coprocessor path with Solidity and Precompiled Contract baselines where appropriate.

5.1 Implementation

Experiments used a Geth-based private Clique network running modified Geth v1.17.5-unstable on a 16-core, 32 GB Aliyun server with Ubuntu 24.04 LTS, built with Go 1.25.7. Solidity contracts were compiled with solc 0.8.26, and WASM modules were built with TinyGo 0.42.0.

Table 1 summarizes the eight contract-facing algorithms, their source or compiled artifact sizes on the three benchmark paths, and their cryptographic roles. Go source (B) denotes the reference implementation source size used by the Precompiled Contract path; WASM bytecode (B) denotes the uploaded module payload compiled from the same reference logic; Solidity runtime (B) denotes the compiled runtime bytecode of the deployed benchmark contract. The Go column therefore reports source size because the Precompiled Contract implementation is statically integrated into the client and does not produce an isolated, per-algorithm deployable artifact.

Upgrade efficiency is measured by comparing the coprocessor upgrade path with Solidity contract deployment using mean setup latency. Execution efficiency

Table 1. Source and compiled artifact sizes of the evaluated algorithms.

Algorithm	Go(B)	WASM(B)	Solidity(B)	Role
Add	156	11,287	172	Non-cryptographic baseline
Sha256	210	68,288	359	Cryptographic hash
PolynomialMul	790	7,987	1,005	Algebraic operation
Blake2bSum256	4,911	8,724	3,722	Cryptographic hash
Pbkdf2Sha256	1,377	72,580	1,556	Key derivation
Dh2048Secret	1,595	45,919	2,490†	Key exchange
PedersenCommit	1,871	95,780	3,094†	Commitment scheme
SchnorrVerify	2,171	95,960	4,233	Signature verification

†Solidity runtime bytecode includes the shared modular-arithmetic library linked at deployment.

is measured by algorithm execution time across EvoCrypt, Solidity, and Precompiled Contract paths after setup. The Precompiled Contract path uses the corresponding client-side implementations summarized in Table 1. Estimated gas denotes the gas estimate for a contract-facing call, not receipt-level `gasUsed`.

5.2 Upgrade Efficiency

Upgrade efficiency was measured by comparing the latency of making an algorithm callable through the coprocessor path with deploying an equivalent Solidity implementation. Figure 3 reports mean setup latency and estimated upgrade gas across all eight algorithms. The Precompiled Contract path is omitted because it requires no separate upload or deployment step in this setup.

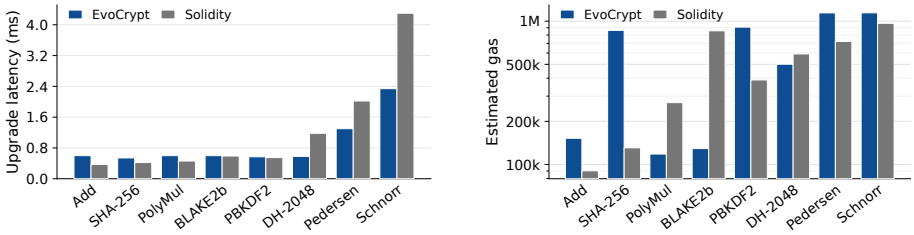


Fig. 3. Upgrade efficiency comparison. Left: mean setup latency; right: estimated gas for EvoCrypt upgrades and Solidity contract deployment.

The two paths were similar for lightweight algorithms. EvoCrypt required less setup time than Solidity for the heavier algorithms in the evaluated suite. For example, Add required 0.60 ms through EvoCrypt and 0.37 ms through Solidity, whereas SchnorrVerify required 2.34 ms and 4.30 ms, respectively. As shown in Figure 3, EvoCrypt incurred slightly higher upgrade gas than Solidity contract deployment in the evaluated setup. These measurements indicate that

the coprocessor can make an algorithm callable without deploying a full Solidity implementation, thereby reducing the development burden of exposing complex cryptography to smart contracts. Its setup path remained comparable to contract deployment for lightweight algorithms.

5.3 Upgrade Consistency

The consistency experiment showed that the evaluated nodes followed the same activation decision after an upgrade was registered, even though event handling and WASM preparation remained node-local. As illustrated in Figure 4, the upgrade transaction records version metadata on chain, nodes prepare the corresponding WASM module, and invocations at or after the Activation Block resolve to the same active version through the canonical version-selection rule. In the evaluated 20-node private Clique network, complete Geth-side upgrade handling finished within 0.54–2.34 ms across the eight algorithms, and the observed activation path produced a common callable version after preparation completed on all evaluated nodes. This supports the intended consistency property of EvoCrypt: local readiness affects preparation timing, whereas version selection is fixed by shared on-chain metadata and block height rather than local event timing. The experiment is nevertheless a controlled private-network validation; it does not measure wide-area propagation, adversarial scheduling, consensus delay under nonzero block periods, or governance failures.

5.4 Execution Efficiency

Steady-state execution cost was evaluated after each algorithm became callable. Figure 5 compares mean algorithm execution time across EvoCrypt, Solidity, and Precompiled Contract paths and estimated gas for EvoCrypt and the Solidity path. Measurements used warm-up runs followed by repeated executions on the private-chain prototype.

The Precompiled Contract path had the shortest algorithm execution time across the evaluated algorithms. EvoCrypt remained close to Solidity on lightweight algorithms, but was faster than Solidity on the heavier cryptographic algorithms while remaining slower than the Precompiled Contract path. For the lightweight set, EvoCrypt execution time was 0.50–0.62 ms. For SchnorrVerify, the corresponding means were 3.67 ms for EvoCrypt, 4.33 ms for Solidity, and 1.26 ms for the Precompiled Contract path. Thus, the relative performance of EvoCrypt depended on algorithm complexity rather than following a single ordering across all algorithms. The reported values are mean algorithm execution times measured after warm-up on the evaluated private-chain prototype.

Estimated gas was compared between the Solidity and EvoCrypt paths to assess contract-facing calling cost under the experimental schedule. SchnorrVerify illustrates the measured difference: the estimate was 2,301,691 gas for the Solidity path and 229,437 gas for EvoCrypt. For lightweight algorithms, the two paths remained within 13–15% of each other. These values show that moving

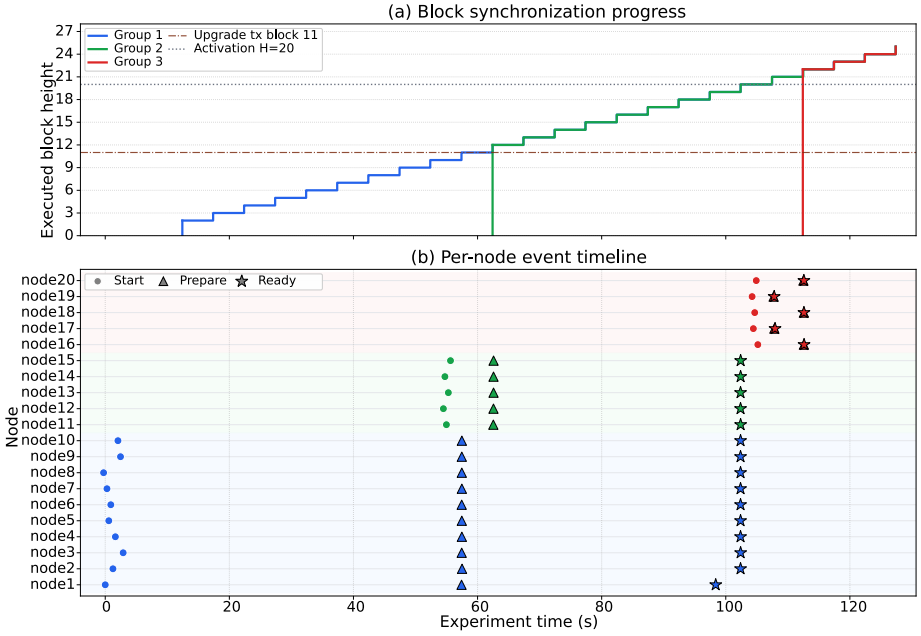


Fig. 4. Consistency of activation-block-based upgrade handling across nodes, including late-joining synchronization, preparation, and ready events.

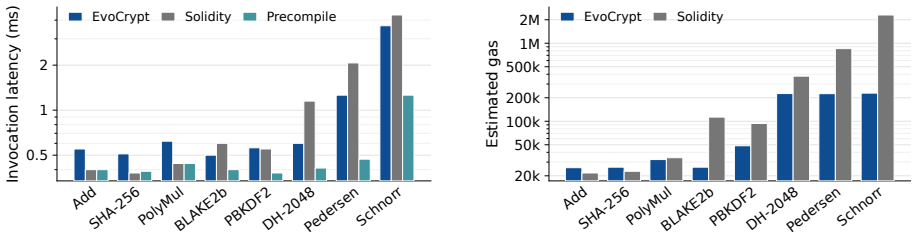


Fig. 5. Execution efficiency comparison. Left: mean algorithm execution time; right: estimated gas for EvoCrypt and Solidity invocations.

complex cryptographic logic out of EVM bytecode can reduce the evaluated calling cost, but they do not establish a calibrated economic schedule or a general magnitude of gas savings. The reported quantity is an estimate, not receipt-level `gasUsed`, and the registered prices are experimental parameters.

Complex-algorithm testing addressed the tension between deployment flexibility and EVM cost. The selected algorithms include post-quantum signatures and pairing-based zero-knowledge proof systems that are substantially more de-

Table 2. Algorithm execution time and uploaded WASM size for computationally intensive cryptographic algorithms.

Algorithm	EvoCrypt (ms)	Precompiled (ms)	WASM (B)
Aigis-Sig2Verify	1.299	1.030	72,213
Dilithium3Verify	1.589	1.100	26,679
ML-DSA-65Verify	1.565	1.052	70,422
SLH-DSA-SHAKE-192fVerify	19.458	6.241	19,339
Groth16Verify/BLS12-381	56.138	6.227	139,279
Groth16Verify/BN254	25.532	3.172	580,975

manding than the regular contract-facing benchmark suite. We evaluated two dimensions: whether the algorithms could be uploaded, prepared, and invoked through a stable interface without a client modification, and what execution cost resulted relative to corresponding Precompiled Contract implementations. Table 2 reports algorithm execution time and uploaded WASM size; times are in milliseconds and module sizes are in bytes.

EvoCrypt executed all algorithms in the stress experiment, including algorithms whose complexity and implementation size place them beyond practical direct Solidity realization in the evaluated setting and that are not present in the evaluated Precompiled Contract catalog. Successful execution supports the deployment-flexibility claim: such algorithms can be made callable through a stable interface without modifying the client for each algorithm. Together with the upgrade mechanism, this result shows how EvoCrypt supports smart-contract-side cryptographic evolution rather than binding contracts to a fixed client-side algorithm catalog.

A clear cost boundary remains. EvoCrypt execution time was 1.3–9.0× the corresponding Precompiled Contract execution time, with the largest gap occurring for Groth16Verify/BLS12-381. EvoCrypt therefore addresses a deployment trade-off rather than providing a universal performance advantage. When a suitable Precompiled Contract already exists, the Precompiled Contract path remains preferable for execution efficiency.

6 Conclusion

EvoCrypt addresses the problem of evolving contract-facing cryptographic capabilities without embedding complex algorithms in EVM bytecode or modifying the client for every update. Its design separates the stable contract-facing interface from replaceable client-side WASM implementations, while deterministic version selection and algorithm-level accounting preserve consistent invocation rules.

Three bounded conclusions follow from the evaluated prototype. EvoCrypt lowers the implementation burden by moving complex cryptographic logic out of Solidity and into replaceable WASM modules, reduces the evaluated calling cost

under the experimental gas schedule, and provides a dynamic upgrade mechanism that lets smart contracts evolve their cryptographic capabilities without changing their contract-facing interface. Algorithm execution remains slower than suitable Precompiled Contracts, and the current gas parameters are not economically calibrated. Future work will refine resource pricing and evaluate broader deployment and security conditions.

References

1. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. https://bitcoin.org/bitcoin.pdf (2008), accessed 21 August 2026
2. Zou, W., Lo, D., Kochhar, P.S., Le, X.B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. IEEE Transactions on Software Engineering **47**(10), 2084–2106 (2021)
3. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. https://ethereum.github.io/yellowpaper/paper.pdf (2014), ethereum Yellow Paper. Accessed 21 August 2026
4. Ethereum.org: Ethereum virtual machine (EVM). https://ethereum.org/developers/docs/evm/, accessed 21 August 2026
5. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM Review **41**(2), 303–332 (1999)
6. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th Annual ACM Symposium on Theory of Computing. pp. 212–219. Association for Computing Machinery, New York, NY, USA (1996)
7. National Institute of Standards and Technology: Post-quantum cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography (2026), updated 5 August 2026. Accessed 21 August 2026
8. EVM Codes: Precompiled contracts. https://www.evm.codes/precompiled, accessed 21 August 2026
9. Input Output: Combinator makes easy work of hard forks. https://iohk.io/en/blog/posts/2020/07/23/combinator-makes-easy-work-of-hard-forks/ (2020), accessed 15 September 2026
10. Cardano Foundation: About hard forks. https://docs.cardano.org/about-cardano/evolution/about-hard-forks, accessed 15 September 2026
11. Ciampi, M., Karayannidis, N., Kiayias, A., Zindros, D.: Updatable blockchains. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) 2020 European Symposium on Research in Computer Security. Lecture Notes in Computer Science, vol. 12309, pp. 590–609. Springer, Cham (2020)
12. Zindros, D.: Soft power: Upgrading chain macroeconomic policy through soft forks. In: Bernhard, M., Bracciali, A., Gudgeon, L., Haines, T., Klages-Mundt, A., Matsuo, S., Perez, D., Sala, M., Werner, S. (eds.) 2021 Financial Cryptography and Data Security International Workshops. Lecture Notes in Computer Science, vol. 12676, pp. 467–481. Springer, Berlin, Heidelberg (2021)
13. Wang, C., Li, P., Fan, X., Tang, Z., Zeng, Y., Sakurai, K.: Phoenix: A live upgradable blockchain client. IEEE Transactions on Sustainable Computing **8**(4), 703–714 (2023)
14. Polkadot: Runtime upgrades. https://docs.polkadot.com/parachains/maintain/runtime-upgrades/, accessed 15 September 2026

15. Rodler, M., Li, W., Karame, G.O., Davi, L.: EVMPatch: Timely and automated patching of ethereum smart contracts. In: Proceedings of the 30th USENIX Security Symposium. pp. 1289–1306. USENIX Association, Berkeley, CA, USA (2021)
16. Kumar, G., Saha, R., Conti, M., Buchanan, W.J.: LEAGAN: A decentralized version-control framework for upgradeable smart contracts. IEEE Transactions on Services Computing **18**(3), 1529–1542 (2025)
17. Reitwiessner, C.: EIP-196: Precompiled contracts for addition and scalar multiplication on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 196. https://eips.ethereum.org/EIPS/eip-196 (2017), final. Accessed 21 August 2026
18. Buterin, V., Reitwiessner, C.: EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 197. https://eips.ethereum.org/EIPS/eip-197 (2017), final. Accessed 21 August 2026
19. Zhang, Y., Crowcroft, J., Li, D., Zhang, C., Li, H., Wang, Y., Yu, K., Xiong, Y., Chen, G.: KylinX: A dynamic library operating system for simplified and efficient cloud virtualization. In: Proceedings of the 2018 USENIX Annual Technical Conference. pp. 173–186. USENIX Association, Berkeley, CA, USA (2018)
20. Pina, L., Andronidis, A., Hicks, M., Cadar, C.: MVEDSUA: Higher availability dynamic software updates via multi-version execution. In: Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 573–585. Association for Computing Machinery, New York, NY, USA (2019)
21. Huang, H., Xiang, C., Zhong, L., Zhou, Y.: PYLIVE: On-the-fly code change for python-based online services. In: Proceedings of the 2021 USENIX Annual Technical Conference. pp. 349–363. USENIX Association, Berkeley, CA, USA (2021)
22. Ajmani, S., Liskov, B., Shrira, L.: Modular software upgrades for distributed systems. In: Thomas, D. (ed.) 2006 European Conference on Object-Oriented Programming. Lecture Notes in Computer Science, vol. 4067, pp. 452–476. Springer, Berlin, Heidelberg (2006)
23. Baresi, L., Ghezzi, C., Ma, X., Panzica La Manna, V.: Efficient dynamic updates of distributed components through version consistency. IEEE Transactions on Software Engineering **43**(4), 340–358 (2017)
24. World Wide Web Consortium: WebAssembly Core Specification. https://www.w3.org/TR/wasm-core-2/, w3C Candidate Recommendation Draft. Accessed 3 September 2026
25. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: Oswald, E., Fischlin, M. (eds.) 2015 Annual International Conference on the Theory and Applications of Cryptographic Techniques, Part II. Lecture Notes in Computer Science, vol. 9057, pp. 281–310. Springer, Berlin, Heidelberg (2015)
26. Rogaway, P.: Formalizing human ignorance: Collision-resistant hashing without the keys. In: Nguyen, P.Q. (ed.) 2006 International Conference on Cryptology in Vietnam. Lecture Notes in Computer Science, vol. 4341, pp. 211–228. Springer, Berlin, Heidelberg (2006)